



UASLP
Universidad Autónoma
de San Luis Potosí

UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

Facultad de Estudios Profesionales Zona Media

PRÁCTICA 6

Encendido de un LED mediante
la pulsación de un botón

Configuración pull-down y lectura de entradas digitales

Alumno: Cesar Gael Mancilla Estrada
Clave: 382031
Carrera: Ingeniería en Mecatrónica
Materia: Microcontroladores
Profesor: Ing. Jesús Padrón
Fecha: 25 de febrero de 2026

“Un botón puede encender una luz, pero también iniciar un sistema completo”

Índice

1. Introducción	2
1.1. Contexto y Justificación	2
1.2. Importancia en la Industria	2
1.3. Objetivos	2
2. Marco Teórico	2
2.1. Configuración Pull-Down	2
2.2. Rebote Mecánico	3
2.3. Funciones del SDK utilizadas	3
3. Desarrollo	3
3.1. Estructura del Proyecto	3
3.2. Archivo: CMakeLists.txt	3
3.3. Archivo: Practica_6.c	4
4. Diagramas del Circuito	5
4.1. Diagrama Esquemático	5
4.2. Diagrama en Protoboard	6
4.3. Descripción de la Conexión	6
5. Proceso de Compilación y Pruebas	7
5.1. Paso a paso	7
5.2. Resultados Esperados	7
6. Preguntas de Repaso	7
6.1. 1. ¿Qué pines del microcontrolador reciben la alimentación externa para no depender del puerto USB?	7
6.2. 2. Explica el funcionamiento de una configuración pull-down en un botón	8
6.3. 3. ¿Qué trabajo cumple la función <code>gpio_init()</code> en el código C?	8
6.4. 4. ¿Qué trabajo cumple la función <code>gpio_get()</code> en el código C?	8
6.5. 5. ¿Qué trabajo cumple la función <code>gpio_put()</code> en el código C?	8
7. Conclusiones	9
7.1. Trabajo Futuro	9
8. Referencias	9

1. Introducción

1.1. Contexto y Justificación

La interacción entre un microcontrolador y el mundo físico no solo implica controlar actuadores (como LEDs o motores), sino también leer señales del entorno mediante sensores y dispositivos de entrada. Un botón o pulsador es el elemento más simple para generar una señal digital que el microcontrolador puede interpretar.

En esta práctica se implementa un sistema donde un LED se enciende únicamente cuando un botón es presionado, utilizando la Raspberry Pi Pico W. Esto introduce conceptos fundamentales como la configuración de pines como entrada, el uso de resistencias pull-down para evitar estados flotantes, y la lectura de señales digitales.

1.2. Importancia en la Industria

La lectura de pulsadores y switches es una habilidad esencial en sistemas embebidos. Desde paneles de control industrial hasta electrodomésticos y dispositivos de interfaz humana (HMI), comprender cómo leer correctamente una entrada digital sin falsas lecturas es crucial para el desarrollo de productos confiables.

1.3. Objetivos

- Configurar un pin GPIO como entrada digital con resistencia pull-down
- Implementar la lectura de un botón usando la función `gpio_get()`
- Controlar un LED como salida en función del estado del botón
- Aprender a alimentar la Raspberry Pi Pico W mediante fuente externa
- Comprender el fenómeno de rebote mecánico y métodos básicos para mitigarlo

2. Marco Teórico

2.1. Configuración Pull-Down

Cuando un pin GPIO se configura como entrada y no está conectado a nada (flotante), puede leer valores indeterminados debido al ruido eléctrico. Para evitar esto, se utiliza una resistencia que fuerza un estado definido:

Tabla 1: Configuraciones de entrada en GPIO

Configuración	Comportamiento
Pull-up	Conecta el pin a 3.3V mediante una resistencia. Estado por defecto = 1
Pull-down	Conecta el pin a GND mediante una resistencia. Estado por defecto = 0
Flotante	Sin resistencia. Estado indeterminado (no recomendado)

En esta práctica se utiliza **pull-down**, por lo que cuando el botón NO está presionado, el pin lee 0. Al presionar el botón, se conecta a 3.3V y el pin lee 1.

2.2. Rebote Mecánico

Los pulsadores mecánicos no hacen contacto instantáneo; al presionarlos o soltarlos, los contactos rebotan generando múltiples transiciones en milisegundos. Esto puede causar múltiples lecturas falsas. La solución básica es agregar un retardo (delay) después de detectar un cambio.

2.3. Funciones del SDK utilizadas

Tabla 2: Funciones para entrada/salida digital

Función	Descripción
<code>gpio_init(pin)</code>	Inicializa el pin
<code>gpio_set_dir(pin, GPIO_IN)</code>	Configura como entrada
<code>gpio_pull_down(pin)</code>	Activa resistencia pull-down
<code>gpio_get(pin)</code>	Lee el valor del pin (0 o 1)
<code>gpio_put(pin, valor)</code>	Escribe valor en pin de salida

3. Desarrollo

3.1. Estructura del Proyecto

Tabla 3: Archivos del proyecto

Archivo	Descripción
<code>Practica_6.c</code>	Código fuente principal
<code>CMakeLists.txt</code>	Configuración de compilación
<code>pico_sdk_import.cmake</code>	Importación del SDK

3.2. Archivo: CMakeLists.txt

Listing 1: Configuración de CMake

```

1 cmake_minimum_required(VERSION 3.13)
2
3 include(pico_sdk_import.cmake)
4 set(PICO_BOARD pico_w)
5
6 project(Practica_6)
7
8 pico_sdk_init()
9
10 add_executable(Practica_6

```

```
11     Practica_6.c
12 )
13
14 target_link_libraries(Practica_6
15     pico_stdlib
16     hardware_gpio
17 )
18
19 pico_enable_stdio_usb(Practica_6 1)
20 pico_enable_stdio_uart(Practica_6 0)
21
22 pico_add_extra_outputs(Practica_6)
```

3.3. Archivo: Practica_6.c

```
1  /*
2   * Practica 6: Encendido de LED con boton
3   * Basado en codigo del Ing. Jesus Padron
4   */
5
6  #include <stdio.h>
7  #include "pico/stdlib.h"
8  #include "hardware/gpio.h"
9
10 #define LED_PIN 3      // GPIO3 (pin 4) para el LED
11 #define BUTTON_PIN 4   // GPIO4 (pin 5) para el boton
12
13 int main() {
14     stdio_init_all();
15
16     printf("Practica 6: LED con boton\n");
17
18     // Inicializar pin del LED como salida
19     gpio_init(LED_PIN);
20     gpio_set_dir(LED_PIN, GPIO_OUT);
21     gpio_put(LED_PIN, 0); // LED apagado inicialmente
22
23     // Inicializar pin del boton como entrada con pull-down
24     gpio_init(BUTTON_PIN);
25     gpio_set_dir(BUTTON_PIN, GPIO_IN);
26     gpio_pull_down(BUTTON_PIN); // Configuracion pull-down
27
28     while (true) {
29         // Leer estado del boton
30         if (gpio_get(BUTTON_PIN)) {
31             // Boton presionado (1 logico)
32             gpio_put(LED_PIN, 1); // Encender LED
33             printf("Boton presionado - LED ENCENDIDO\n");
34         } else {
35             // Boton no presionado (0 logico)
36             gpio_put(LED_PIN, 0); // Apagar LED
37             printf("Boton liberado - LED APAGADO\n");
38         }
39
40         sleep_ms(100); // Pequena pausa para evitar rebotes
41     }
```

```
42  
43     return 0;  
44 }
```

Listing 2: Código completo - LED controlado por botón

4. Diagramas del Circuito

4.1. Diagrama Esquemático

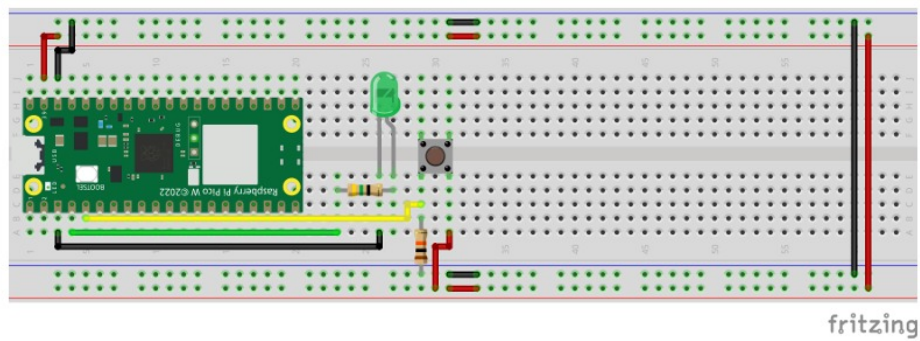


Figura 1: Diagrama esquemático de la conexión del LED y botón a la Raspberry Pi Pico W

4.2. Diagrama en Protoboard

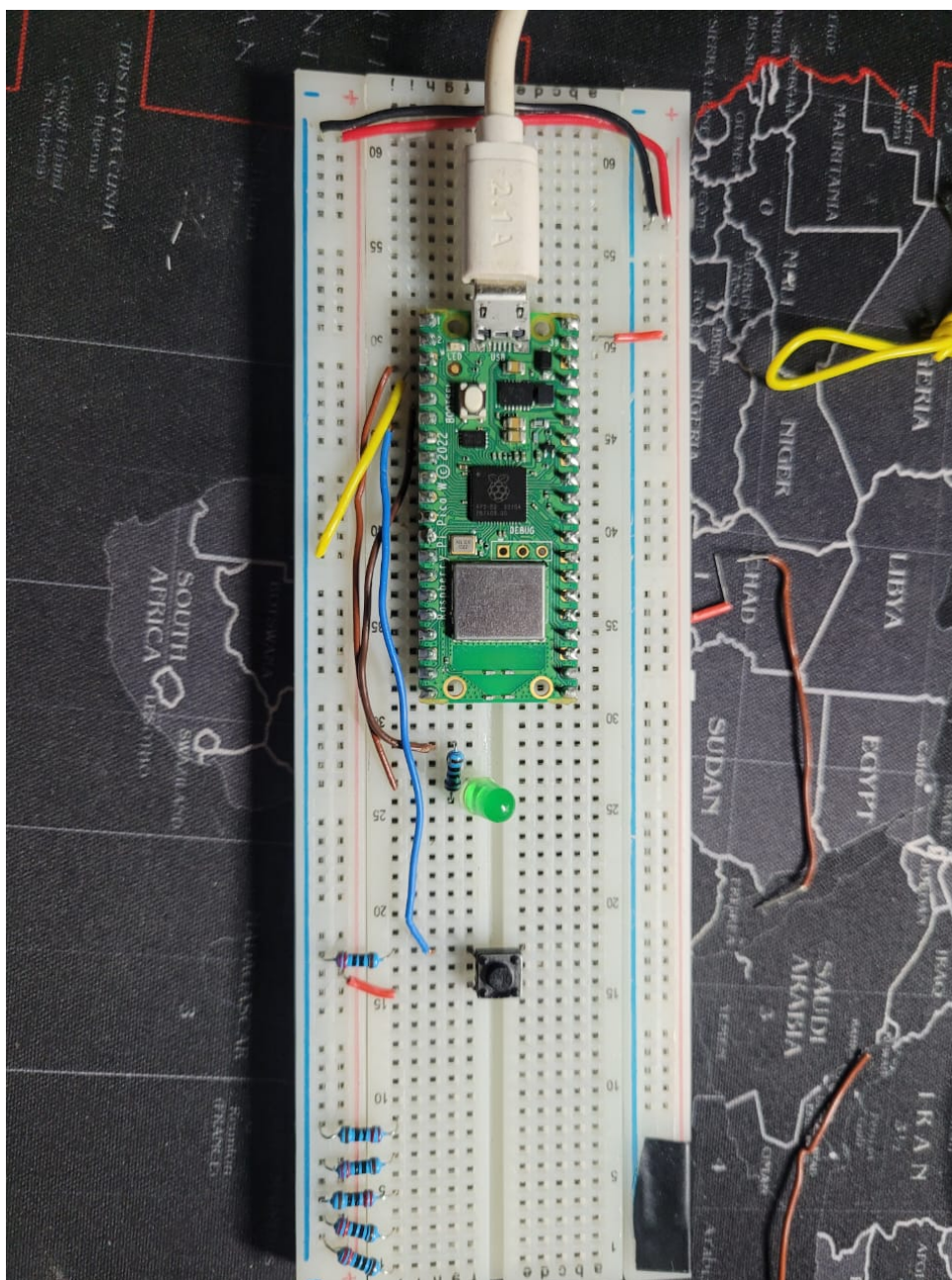


Figura 2: Conexión del LED y botón en protoboard con alimentación externa

4.3. Descripción de la Conexión

- **LED:** Ánodo (terminal larga) conectado a **GPIO3** (pin 4) a través de una resistencia de 68Ω . Cátodo a GND.
- **Botón:** Un terminal conectado a **GPIO4** (pin 5). El otro terminal a **3.3V**.
- **Resistencia pull-down interna:** Activada por software mediante `gpio_pull_down()`. Cuando el botón no está presionado, GPIO4 se mantiene en 0V.
- **Alimentación externa:** La placa puede alimentarse mediante el pin **VBUS** (pin 40) con 5V, o mediante **VSYS** (pin 39) con 3.3V - 5.5V.

5. Proceso de Compilación y Pruebas

5.1. Paso a paso

1. Se creó la carpeta `Práctica_6` con los archivos `Practica_6.c`, `CMakeLists.txt` y `pico_sdk_import.cmake`
2. Se abrió la carpeta en Pico - Visual Studio Code
3. Se seleccionó el kit Pico ARM GCC
4. Se compiló el proyecto, generando `Practica_6.uf2`
5. Se cargó el programa a la Pico W mediante modo BOOTSEL
6. Se armó el circuito en protoboard
7. Se alimentó la placa mediante USB o fuente externa
8. Se verificó que al presionar el botón, el LED se enciende

5.2. Resultados Esperados

Práctica 6: LED con botón

Botón liberado - LED APAGADO

Botón liberado - LED APAGADO

Botón presionado - LED ENCENDIDO

Botón presionado - LED ENCENDIDO

Botón liberado - LED APAGADO

6. Preguntas de Repaso

6.1. 1. ¿Qué pines del microcontrolador reciben la alimentación externa para no depender del puerto USB?

La Raspberry Pi Pico W puede ser alimentada externamente a través de los siguientes pines:

- **VBUS (pin 40):** Acepta 5V provenientes de un cargador USB o fuente de 5V.
- **VSYS (pin 39):** Acepta de 3.3V a 5.5V. Es la entrada principal de alimentación.

Para usar alimentación externa, se aplica voltaje a VSYS (o VBUS si es 5V) y se conecta GND (pin 38). El regulador interno de la Pico W genera los 3.3V necesarios para el RP2040.

6.2. 2. Explica el funcionamiento de una configuración pull-down en un botón

La configuración pull-down utiliza una resistencia (interna o externa) que conecta el pin GPIO a GND (0V).

Funcionamiento:

- **Botón no presionado:** El pin está conectado a GND a través de la resistencia pull-down. El microcontrolador lee 0 lógico (APAGADO).
- **Botón presionado:** El botón conecta el pin directamente a 3.3V. La resistencia pull-down es mucho más grande (típicamente 10kΩ) que el camino a 3.3V (casi 0Ω), por lo que el pin lee 1 lógico (ENCENDIDO).

En el código, se activa con `gpio_pull_down(BUTTON_PIN)`.

6.3. 3. ¿Qué trabajo cumple la función `gpio_init()` en el código C?

`gpio_init(pin)` inicializa el pin GPIO especificado para su uso. Esta función debe llamarse antes de cualquier otra operación con ese pin. Habilita el pin y lo prepara para ser configurado como entrada o salida.

6.4. 4. ¿Qué trabajo cumple la función `gpio_get()` en el código C?

`gpio_get(pin)` lee el estado actual de un pin GPIO configurado como entrada. Retorna:

- **1** si el pin detecta un nivel alto (aproximadamente >1.8V)
- **0** si el pin detecta un nivel bajo (aproximadamente <0.8V)

En esta práctica, se usa para saber si el botón está presionado.

6.5. 5. ¿Qué trabajo cumple la función `gpio_put()` en el código C?

`gpio_put(pin, valor)` escribe un valor en un pin configurado como salida:

- `gpio_put(pin, 1)` → Establece el pin en 3.3V (alto)
- `gpio_put(pin, 0)` → Establece el pin en 0V (bajo)

En esta práctica, se usa para encender o apagar el LED.

7. Conclusiones

- Se aprendió a configurar un pin GPIO como entrada digital utilizando la función `gpio_set_dir(pin, GPIO_IN)` y a activar la resistencia pull-down interna con `gpio_pull_down()`.
- Se implementó la lectura de un botón mediante `gpio_get()`, permitiendo controlar el estado de un LED de forma condicional.
- Se comprendió la importancia de las resistencias pull-up/pull-down para evitar estados flotantes en pines de entrada.
- Se identificaron los pines de alimentación externa (VSYS y VBUS) para operar la Pico W sin depender del puerto USB.
- Se aplicó un retardo de 100 ms para mitigar el efecto del rebote mecánico del pulsador.

7.1. Trabajo Futuro

- Implementar un anti-rebote por software más robusto usando temporizadores
- Controlar el brillo del LED con PWM mientras el botón está presionado
- Usar interrupciones GPIO en lugar de polling continuo
- Agregar un segundo botón para cambiar el patrón de encendido

8. Referencias

- [1] Raspberry Pi Foundation. (2024). *Raspberry Pi Pico W Datasheet*. Sección 3.5: Powering.
- [2] Raspberry Pi Foundation. (2024). *Pico SDK Documentation*.
- [3] RP2040 Datasheet. (2024). *RP2040: A microcontroller from Raspberry Pi*.